

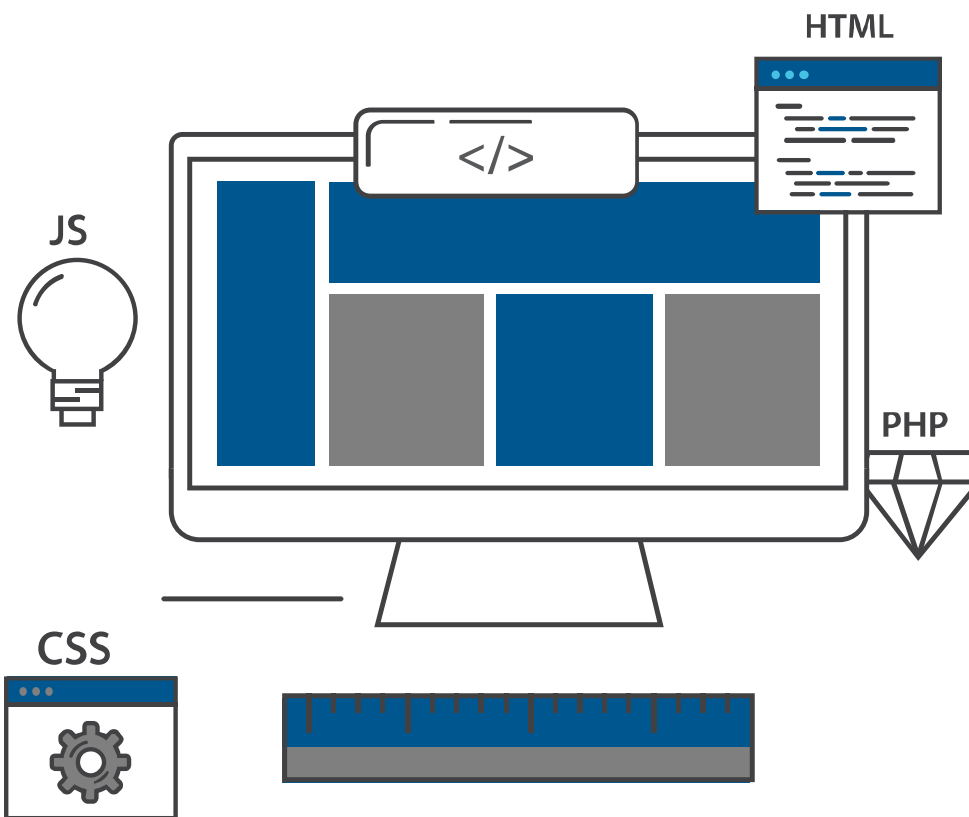
第二章 算法分析基础

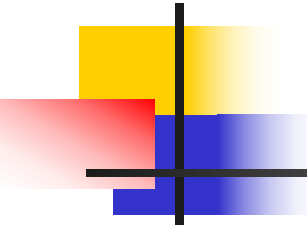
李翔

<https://implus.github.io/>



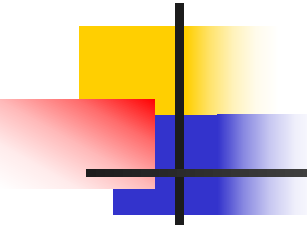
C++





定义效率

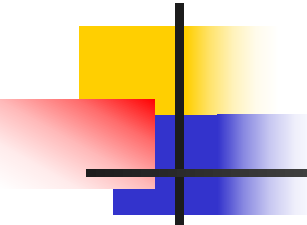
- 寻找效率的定义：与平台无关，实例无关，并且随着输入规模的增长是可以预测的



定义效率

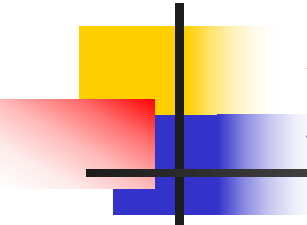
Table 2.1 The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds 10^{25} years, we simply record the algorithm as taking a very long time.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long



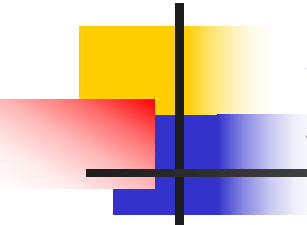
定义效率

一个算法被称为是**多项式时间的**如果满足如下的性质：
当算法输入的规模增长时，算法的运行时间是多项式有界的。也就是：存在常数 $c > 0$, $d > 0$, 使得对于每一个问题输入的规模 N , 算法的运行都能够在 cN^d 步骤内完成。



复杂度符号

- 因为上述原因，希望以不受常数因子，低项影响的方式表达运行时间的增长率。
- $O, \Omega, \dots$



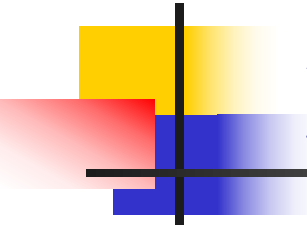
复杂度符号

- **Upper bounds.** $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \leq c \cdot f(n)$.
- **Lower bounds.** $T(n)$ is $\Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \geq c \cdot f(n)$.
- **Tight bounds.** $T(n)$ is $\Theta(f(n))$ if $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$.



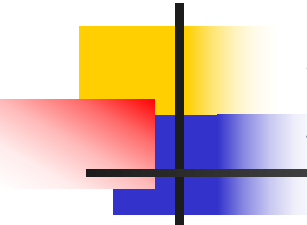
复杂度符号

- Ex: $T(n) = 32n^2 + 17n + 32$.
 - $T(n)$ is $O(n^2)$, $O(n^3)$, $\Omega(n^2)$, $\Omega(n)$, and $\Theta(n^2)$.
 - $T(n)$ is not $O(n)$, $\Omega(n^3)$, $\Theta(n)$, or $\Theta(n^3)$.



复杂度性质

- 设 f 和 g 是两个函数, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在, 并且等于某个常数 $c > 0$, 那么 $f(n) = \Theta(g(n))$.
- 证明: 根据极限的定义, 存在 n_0 , 对所有的 $n \geq n_0$, $f(n) \leq 2cg(n)$ 从而推出 $f(n) = O(g(n))$. 类似的 $f(n) = \Omega(g(n))$. 根据定义可知命题成立。

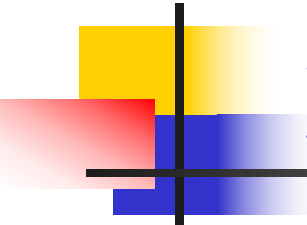


复杂度性质

■ 传递性

- If $f = O(g)$ and $g = O(h)$ then $f = O(h)$.
- If $f = \Omega(g)$ and $g = \Omega(h)$ then $f = \Omega(h)$.
- If $f = \Theta(g)$ and $g = \Theta(h)$ then $f = \Theta(h)$.

本质是：不等号的传递性!

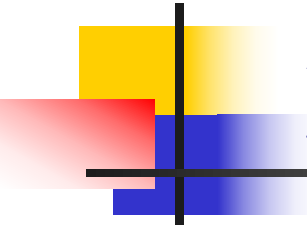


复杂度性质

■ 可加性

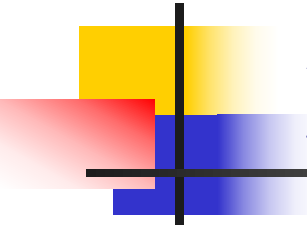
- If $f = O(h)$ and $g = O(h)$ then $f + g = O(h)$.
- If $f = \Omega(h)$ and $g = \Omega(h)$ then $f + g = \Omega(h)$.
- If $f = \Theta(h)$ and $g = \Theta(h)$ then $f + g = \Theta(h)$.

本质是：不等号的可加性!



复杂度性质

- 命题 令 f 是一个 d 阶多项式, $a_0 + a_1n + \dots + a_d n^d$, $a_d > 0$, 那么 $f = \Theta(n^d)$.
- 命题 对于任何正整数, $O(\log_a n) = O(\log_b n)$.



复杂度性质

- 命题 对于 $x > 0$, $\log n = O(n^x)$.

对数函数增长比多项式增长慢!

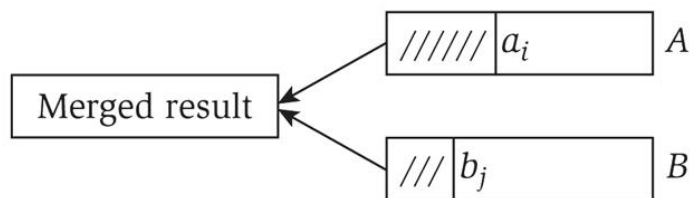
- 命题 对每个 $r > 1$ 和每个 $d > 0$, $n^d = O(r^n)$.

多项式增长比指数函数慢!

线性时间

归并两个排好序的数表

- 问题：给定两个数组，每个包含有 n 个数，并且每个按照上升的顺序排列， $\mathbf{A} = \mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n$ ； $\mathbf{B} = \mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ ，把它们归并成按照上升顺序排列的数组。

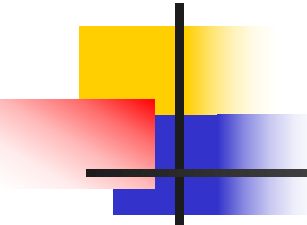


```
i = 1, j = 1
while (both lists are nonempty) {
    if (a_i ≤ b_j) append a_i to output list and increment i
    else (a_i > b_j) append b_j to output list and increment j
}
append remainder of nonempty list to output list
```



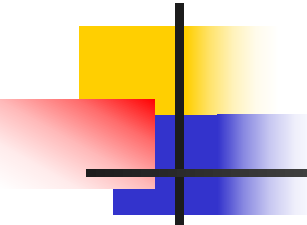
$O(n\log n)$ 阶时间

- 快速排序(Quicksort)
- FFT(快速傅立叶变换)
- 归并排序, 堆排序



$O(n^k)$ 时间

- 问题： 对某个固定常数 k , 我们想知道给定的 n 个结点的输入图是否有一个大小为 k 的独立集。
- 如果用蛮力搜索的方法： 枚举所有 k 个结点的子集，并且对每个子集 S 检查是否存在一条边与 S 中的任意两个元素相交



$O(n^k)$ 时间

```
foreach subset S of k nodes {  
    check whether S is an independent set  
    if (S is an independent set)  
        report S is an independent set  
    }  
}
```

枚举所有 k 个结点的子集: $\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots(2)(1)} \leq \frac{n^k}{k!}$

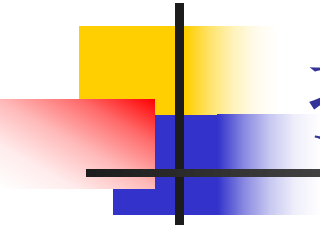
判断 k 个结点的子集是否独立: $O(k^2)$

所以蛮力搜索的代价为: $O(k^2 n^k / k!) = O(n^k)$



指数时间(Exponential time)

- 问题： 假设给定一个图， 需要找一个**最大规模的独立集**。
- 解决思路： 蛮力搜索的方法， 检查每一个子集是否是独立集， 是否最大

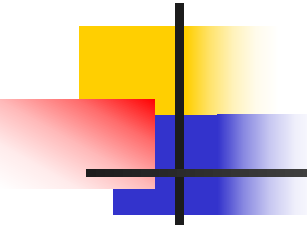


指数时间(Exponential time)

```
S* ←  $\phi$ 
foreach subset S of nodes {
    check whether S is an independent set
    if (S is largest independent set seen so far)
        update S* ← S
    }
}
```

估算计算时间的代价

$$O(n^2 2^n)$$



n!时间

- 对稳定匹配问题的穷举搜索: $n!$
- 二分匹配(二部图每边存在 n 个结点)问题
如果穷举搜索, 代价是 $n!$
- 巡回售货员(TSP: Travelling Salesman Problem)问题: 给定 n 个城市的集合, 每对城市之间都有距离, 什么是访问所有城市的最短旅行? 固定第一个(结束)城市
穷举代价是 $(n-1)!$